◉ تقرير تدقيق شامل · 2026

AcademyBase
Full Product & Technical Audit

تحليل معمّق لبنية النظام، تدفق البيانات، سلامة المنطق، تجربة المستخدم، والأداء — مع توصيات مُرتّبة حسب الأولوية لتحويل المنصة إلى منتج SaaS متميّز.

المنصة	التقنيات	تاريخ التقرير	نوع التدقيق
AcademyBase SaaS	Next.js 16 · Supabase · TypeScript	مارس 2026	تقني · منتج · UX

10	12	7	7
نقاط ضعف تقنية	مشكلة في تجربة المستخدم	مخاطر في منطق البيانات	مخاطر الأداء

محتويات التقرير

01 نظرة عامة على النظام
02 نقاط الضعف الرئيسية
03 مشكلات تجربة المستخدم
04 مخاطر البيانات والمنطق
05 مخاطر الأداء والقابلية للتوسع
06 التحسينات عالية الأثر
07 التوصيات الاستراتيجية
— جدول الأولويات

01 نظرة عامة على النظام

كيف تترابط الصفحات والمكونات والوحدات — وتدفق البيانات بين جميع أجزاء النظام

بنية البيانات الأساسية

```
academies └ branches (price, days[], subscription_mode, rent_type, monthly_rent) ├ players
(subscription_mode, sessions, start_date, end_date, is_paused) ├ staff (monthly_salary,
branch_ids[], assign_mode) ├ calendar_events (training/match/canceled/special_event,
deduct_sessions) ├ sessions (status: scheduled/cancelled/completed — finance source of truth)
├ attendance (player daily presence) └
staff_attendance(present/late/absent/vacation/excused/no_training) payments (player_id, kind:
new/renew/legacy, subscription_end) finance_tx (auto_key, source, branch_id TEXT, category,
amount) insights (pure computation — no DB table, derived from all above)
```

تدفق البيانات عبر الوحدات — سلسلة إلغاء التدريب

تُعدّ سلسلة إلغاء التدريب من التقويم الأكثر تعقيدًا في النظام. عند إلغاء جلسة تدريبية واحدة، تُنفَّذ **6 عمليات غير متزامنة** بشكل متتابع:

```
1. upsertSession() → sessions.status = "cancelled" 2. syncSessionFieldFinance() →
suppresses/zeroes field cost in finance_tx 3. syncRentAggregateForBranch() → recalculates
monthly rent aggregate 4. syncCoachDeductionForSession() → markNoTrainingForBranch() →
staff_attendance rows 5. syncSalaryFinanceEntry() → recomputes net salary per staff in
finance_tx 6. extendBranchPlayersByOneSession() → +1 session (or +N days) per player
```

⚠️ **لا توجد معاملة (Transaction) تلفّ هذه العمليات الست.** إذا فشلت أي خطوة في المنتصف، يبقى النظام في حالة غير متسقة دون أي تراجع أو إشعار للمستخدم.

المدفوعات ← المالية

إنشاء دفعة في صفحة اللاعبين يستدعي `upsertAutoFinanceTx()` بـ `auto_key = "revenue:YYYY-MM:branchId"`. صفحة المالية تقرأ هذا الإدخال. مدفوعات `kind: "legacy"` مستثناة من التحليلات.

حضور الطاقم ← المالية

صفحة حضور الطاقم تحسب الخصومات عبر `computeSessionDeduction()`، ثم `syncSalaryFinanceEntry()` في sessions.ts تسحب سجلات الحضور الحية لإعادة حساب صافي الراتب في finance_tx.

📊 الرؤى ← لوحة التحكم

`computeInsights()` دالة نقية تأخذ بيانات اللاعبين، المدفوعات، الحضور، الفروع، والمالية. لوحة التحكم تمرر `attendance=[]` مما يُصمّت رؤية الغيابات المتتالية دائمًا.

📅 التقويم ← الجلسات

`calendar_events` هي الطبقة المرئية للمستخدم، بينما `sessions` هي مصدر الحقيقة لمزامنة المالية. الجدولان يمكن أن يتباعدا: حذف صف في sessions لا يحذف حدث التقويم المقابل.

⚡ طبقة التخزين المؤقت (Module-Level Cache)

النظام يستخدم ثلاثة تخزينات مؤقتة على مستوى الوحدة (process-level)، تُمسح عند تسجيل الخروج:

المتغير	المحتوى	يُمسح عند
`_academyIdCache`	Promise<string> لـ academy_id	تسجيل الخروج
`_membershipCache`	Promise<MembershipInfo> (الدور، الصلاحيات)	تسجيل الخروج
`_sidebarCache`	SidebarData (اسم الأكاديمية، المستخدم، الشعار)	تسجيل الخروج + clearSidebarCache()

02 نقاط الضعف الرئيسية

إشكاليات تقنية جوهرية تؤثر على سلامة النظام واستقراره

W1 لا توجد معاملة (Transaction) حول سلسلة إلغاء التدريب حرج

الـ 6 عمليات المترتبة على إلغاء جلسة تدريبية (تحديث الجلسة، المالية، تعويض اللاعبين، خصومات الطاقم) لا تُغلَّف بأي معاملة ذرية. إذا فشلت الخطوة الرابعة بعد نجاح الثلاث الأولى، ينتهي النظام في حالة متناقضة: تكلفة الملعب محذوفة، لكن رواتب الطاقم وجلسات اللاعبين لم تُحدَّث. لا توجد آلية تراجع أو إعادة محاولة.

W2 تحديث اللاعبين يعمل بحلقة O(n) بدل استعلام واحد متوسط

`extendBranchPlayersByOneSession()` تجلب كل لاعبي الفرع ثم تنفّذ `updatePlayer(id, ...)` لكل واحد منهم عبر `Promise.all`. لفرع يضم 80 لاعبًا، هذا 80 استعلام UPDATE متوازٍ. كان يجب أن يكون استعلامًا واحدًا: `UPDATE players SET sessions = sessions + 1 WHERE branch_id = $1`. والأشد خطورة: الدالة تُعوّض اللاعبين الموقوفين والمنتهية اشتراكاتهم بنفس القدر.

W3 عمود `finance_tx.branch_id` من نوع TEXT لا UUID مع Foreign Key حرج

لا يوجد قيد FK بين finance_tx وجدول branches. عند حذف فرع، تبقى جميع سجلاته المالية التاريخية يتيمة — لا يمكن إعادة ربطها باسم الفرع. الواجهة قد تعرض UUID خامًا أو حقلًا فارغًا، والسجل التاريخي تالف بشكل دائم.

W4 لوحة التحكم تُمرّر `attendance=[]` لدالة الرؤى متوسط

`computeInsights()` في لوحة التحكم لا تحصل على بيانات الحضور، مما يجعل رؤية "الغيابات المتتالية" — الأكثر قيمة تشغيليًا — صامتة دائمًا على لوحة التحكم. صفحة الرؤى فقط تعرضها بشكل صحيح لأنها تجلب بياناتها بشكل مستقل.

W5 فجوة في إبطال كاش الشريط الجانبي بعد رفع الشعار منخفض

عند رفع شعار الأكاديمية، يُستدعى `clearSidebarCache()` لكن مكوّن Sidebar مثبّت حاليًا في الصفحة. مسح الكاش لا يُعيد تشغيل `useEffect`. الشعار يظهر فورًا في صفحة الإعدادات (local state) لكن يبقى الشريط الجانبي على الصورة القديمة حتى يتنقل المستخدم لصفحة أخرى.

W6 تداخل بين `calendar_events` و `sessions` في تمثيل نفس البيانات متوسط

التقويم المرئي يستخدم `calendar_events`، بينما مزامنة المالية تعتمد على `sessions`. جلسة ملغاة تنشئ صفًا في كلا الجدولين، ويمكن أن يتباعدا: حذف صف sessions لا يحذف حدث التقويم، وصفحة حضور اللاعبين تقرأ calendar_events لتوليد تواريخ إضافية متجاوزةً جدول sessions كليًا.

W7 لا يوجد حد أدنى لخصم الجلسات — الجلسات قد تصير سالبة حرج

`reduceBranchPlayersByOneSession()` تطرح جلسة واحدة من كل لاعب في الفرع عند استعادة جلسة ملغاة، دون أي تحقق من أن `sessions >= 1`. لاعب لديه صفر جلسات سيُحوَّل إلى -1. لا يوجد CHECK constraint على مستوى قاعدة البيانات.

W8 لوحة تشخيص المطوّرين موجودة في كود الإنتاج منخفض

صفحة الفروع تحتوي على `showDiagnostics` state toggle يعرض مخرجات تقنية خام (UUIDs، نتائج RLS، كائنات Supabase الخام). أي مستخدم حقيقي يضغطها سيواجه ناتجًا محيّرًا. يجب إزالتها أو حصرها بـ development environment فقط.

W9 معالجة NULL في `subscription_end` غير دقيقة للسجلات التاريخية متوسط

للمدفوعات قبل migration 16 (التي أضافت عمود subscription_end)، يحسب النظام تاريخ الانتهاء بالرجوع إلى `player.end_date`، الذي يُكتب فوق قيمته عند كل تجديد. لاعب جدّد اشتراكه 3 مرات قبل migration 16 لا يمكن للنظام إعادة بناء الفترات التاريخية الدقيقة لأول وثاني تجديد. حالات الحضور للاعبين القدامى تبقى تقريبية لا دقيقة.

W10 `countSessionsInMonth` **تُعاد حسابها لكل عضو طاقم في كل render** منخفض

الدالة تتكرر على كل أيام الشهر للتحقق من أيام الفرع في كل مرة تُستدعى. في صفحة حضور الطاقم، تُستدعى داخل منطق عرض كل خلية. لفرع فيه 20 موظفًا وشهر مدته 30 يومًا، تنفّذ مئات المرات بنفس المدخلات دون أي memoization.

03

مشكلات تجربة المستخدم

نقاط احتكاك تؤثر على صاحب الأكاديمية والمدير في الاستخدام اليومي

UX1 — لا يوجد P&L لكل فرع

عالي الأثر

صفحة المالية تُجمّع المعاملات حسب التصنيف لا حسب الفرع. مالك أكاديمية بـ 4 فروع لا يستطيع معرفة أي فرع مربح دون جمع يدوي. هذا أكثر سؤال شائع لمالك متعدد الفروع.

UX2 — إلغاء التدريب بدون معاينة العواقب

عالي الأثر

عند النقر على "إلغاء التدريب"، لا يُخبر النظام المستخدم بما سيحدث: "سيُعوَّض 47 لاعبًا بجلسة إضافية، وسيُخصم من راتب 6 مدربين." القرار مصيري لكنه يُتخذ بدون معاينة تأثيره.

UX3 — الرؤى للقراءة فقط — لا مسار للتصرف

متوسط

صفحة الرؤى تعرض "7 لاعبين تنتهي اشتراكاتهم خلال 3 أيام" لكن لا يوجد زر لتجديدهم أو الاتصال بهم. حقل `actions[]` موجود في نوع البيانات لكنه يُعرض كنص فقط لا كإجراءات قابلة للنقر.

UX4 — البحث عن اللاعبين غير محسوب عند التوسع

متوسط

`listPlayers()` تجلب كل اللاعبين عند كل تحميل للصفحة. الفلترة تتم بالكامل في React state عبر `.filter()`. لأكاديمية بـ 400+ لاعب، التحميل الأولي بطيء ولا يوجد pagination أو debounce على حقل البحث.

UX5 — لا يوجد سجل مدفوعات للاعب في صفحة اللاعبين

متوسط

من صفحة اللاعبين يمكن رؤية تاريخ انتهاء الاشتراك الحالي فقط. لمعرفة تاريخ المدفوعات أو انتظام الدفع، يجب الانتقال يدويًا لصفحة المالية والبحث هناك. الدالة `listPlayerPayments()` موجودة في الكود لكن لا واجهة لها في صفحة اللاعبين.

UX6 — صفحة الطاقم تُظهر الراتب الإجمالي فقط

متوسط

لا يوجد عرض للخصومات أو صافي الراتب أو ملخص الحضور في صفحة الطاقم. الراتب الفعلي بعد الغيابات موجود فقط في finance_tx، ولا يمكن رؤيته إلا بالذهاب لصفحة المالية والبحث يدويًا.

UX7 — لوحة التحكم "هذا الشهر" ثابتة

منخفض

KPIs لوحة التحكم محسوبة دائمًا للشهر الحالي. لا يوجد سهم "الشهر السابق" أو محدد نطاق زمني. لمراجعة أداء الشهر الماضي، يجب الذهاب للإحصائيات أو المالية.

UX8 — لا يمكن حذف شعار الأكاديمية

منخفض

بعد رفع الشعار، لا يوجد زر "إزالة الشعار" للعودة إلى الحرف الأول. زر "رفع شعار" سيستبدله، لكن الواجهة لا توضح ذلك، ولا توجد طريقة لإزالته نهائيًا.

UX9 — دور مدير الفرع مُطبَّق جزئيًا

متوسط

دور `branch_manager` موجود في قاعدة البيانات مع عمود `has_finance_access` في academy_members، لكن لا توجد واجهة في الإعدادات لتعديل هذا الإعداد. المالك لا يستطيع منح/سحب صلاحية المالية لمدير فرع من داخل التطبيق.

UX10 — التقويم لا يُظهر نسبة حضور كل تدريب

منخفض

شبكة التقويم تُظهر نقاطًا ملوّنة لأنواع الأحداث، لكن لا مؤشر على "24 من أصل 30 لاعبًا حضروا." كل أيام التدريب تبدو متطابقة بصريًا بغض النظر عن جودة الحضور.

UX11 — الحالات الفارغة غائبة في معظم الصفحات

منخفض

صفحة الإحصائيات بدون بيانات تُظهر مخططات فارغة. صفحة المالية بدون معاملات تُظهر جدولًا فارغًا. لا توجد رسائل توجيهية ولا دعوة للإجراء مثل "أضف أول فرع للبدء."

UX12 — لا عمليات مجمّعة على اللاعبين

منخفض

لا يمكن تجديد، إيقاف، أو نقل عدة لاعبين دفعة واحدة. كل عملية تتطلب فتح modal لكل لاعب على حدة. لأكاديمية تجدد بداية كل شهر، هذا احتكاك يومي ملموس.

04 مخاطر البيانات والمنطق

تناقضات وأخطاء منطقية قد تؤدي إلى بيانات غير صحيحة أو تالفة

تحذير: بعض هذه المخاطر تؤدي إلى تلف صامت في البيانات — المستخدم لا يرى أي خطأ، لكن الأرقام تصبح غير صحيحة.

R1 تعويض الجلسات يشمل اللاعبين الموقوفين والمنتهية اشتراكاتهم حرج

`extendBranchPlayersByOneSession(branchId, branchDays)` تجلب اللاعبين بـ `branch_id = branchId` فقط — بدون أي فلتر على `is_paused` أو حالة الاشتراك. لاعب منتهي اشتراكه منذ شهرين ولاعب موقوف يحصلان على جلسة إضافية لن يستخدمانها أبدًا. للاعبي الاشتراك الشهري، تُضاف أيام إضافية لتاريخ انتهاء قد يكون منتهيًا أصلًا، مما يُعيد تنشيطه بشكل زائف في شبكة الحضور.

R2 `daysPerSession = 30 / sessionsInMonth` ينتج Infinity عند غياب أيام التدريب حرج

إذا أُنشئ فرع بدون تحديد أيام التدريب، فإن `countSessionsInMonth()` تُعيد 0، و `daysPerSession = 30 / 0 = Infinity`. تطبيق `addDays(end_date, Infinity)` على تاريخ ينتج `Invalid Date`، والذي يُكتب إلى `players.end_date` كقيمة تالفة. لا يوجد تحقق للحماية من هذا السيناريو.

R3 خصومات الراتب تقسم على `branchCount` الكلي بما فيها الفروع المحذوفة متوسط

في `syncSalaryFinanceEntry()`، نصيب الراتب يُحسب بـ `monthly_salary / branchCount` حيث `branchCount = staff.branch_ids.length`. إذا كان الموظف مسجلًا في 3 فروع وحُذف أحدها، قد يبقى UUID الفرع المحذوف في مصفوفة `branch_ids` (لا يوجد CASCADE على مستوى التطبيق)، مما يجعل الراتب يُقسَّم على 3 بدلًا من 2 — خصم منهجي غير صحيح.

R4 `listPaymentPeriods()` غير مستخدمة في لوحة التحكم والرؤى متوسط

صفحة الحضور تستخدم بشكل صحيح `listPaymentPeriods()` مع عمود `subscription_end` لتحديد الفترات النشطة. لوحة التحكم والرؤى لا تزال تستخدم مصفوفة المدفوعات الخام وتشتق حالة الاشتراك من `player.end_date`. النظامان لديهما فهم مختلف لحالة اشتراك اللاعب.

R5 إدخالات `no_training` من التقويم قد تُكتب فوق إدخالات يدوية متوسط

إلغاء تدريب من التقويم يُنشئ سجلات `no_training` في staff_attendance عبر `markNoTrainingForBranch()`. إذا فتح مدير لاحقًا صفحة حضور الطاقم وغيّر حالة يدويًا، يستخدم `upsertStaffAttendance()` الذي يُضاربها. علم `deduct_from_salary` قد يختلف بين الإدخالين، مؤثرًا على حسابات الراتب النهائية.

R6 تصنيف "new vs renew" للمدفوعات يمكن أن يتعارض بين نقطتي حساب متوسط

`createPayment()` في صفحة اللاعبين تُحدد `kind: "new"` أو `"renew"` بناءً على ما إذا كان `start_date` يتطابق مع اليوم الحالي. لكن `renewalStats` في لوحة التحكم تُعيد اشتقاق التصنيف من مصفوفة المدفوعات بالتحقق من وجود دفعات سابقة. بيانات مُهاجَرة بشكل خاطئ ستُصيب معدل التجديد المحسوب في لوحة التحكم.

R7 `syncSalaryFinanceEntry` يعتمد على قراءة سجلات الحضور في الوقت الفعلي متوسط

في كل مرة تُستدعى هذه الدالة، تجلب كل سجلات حضور الطاقم للشهر وتعيد حساب صافي الراتب من الصفر. لا يوجد snapshot للحسابات. إذا حُذف سجل حضور تاريخيًا، تتغير قيمة finance_tx المقابلة عند أي استدعاء تالٍ للدالة، حتى لأشهر مضت وأُغلق حسابها.

05 مخاطر الأداء والقابلية للتوسع

مناطق ستتباطأ أو تنهار مع نمو حجم البيانات

#	المشكلة	المكان	الحجم الحرج	الخطورة
P1	لوحة التحكم تجلب كل اللاعبين وكل المدفوعات (كل التاريخ) عند كل تحميل	`dashboard/page.tsx`	+500 لاعب	عالي
P2	صفحة الإحصائيات تجلب كل سجلات الحضور لـ 12 شهرًا دفعة واحدة من الجانب العميل	`statistics/page.tsx`	+96,000 سجل	عالي
P3	تعويض اللاعبين: N استعلام UPDATE متوازٍ بدل استعلام واحد	`players.ts`	+100 لاعب	متوسط
P4	صفحة التقويم هي الأثقل في التطبيق — تجلب فروع، طاقم، جلسات، أحداث، مدفوعات، لاعبين دفعة واحدة	`calendar/page.tsx`	بيانات متوسطة	متوسط
P5	صفحة الرؤى تُكرّر كل عمليات جلب البيانات بشكل مستقل عن لوحة التحكم — لا كاش مشترك	`insights/page.tsx`	أي حجم	متوسط
P6	صفحة المالية بنطاق 12 شهرًا قد ترجع آلاف السجلات مع تجميع عميل كامل	`finance/page.tsx`	+500 سجل	متوسط
P7	جلب كل فترات الدفع لكل اللاعبين في صفحة الحضور بدون فلتر تاريخ	`listPaymentPeriods()`	+300 لاعب	متوسط

📌 **الجذر المشترك لمشاكل الأداء:** جميع الصفحات الرئيسية هي Client Components تُطلق جلب البيانات بعد الـ mount (blank → loading → data). لا توجد Server Components أو Suspense boundaries. التحول لـ React Server Components سيحل معظم هذه المشاكل دفعة واحدة.

06 التحسينات عالية الأثر

مُرتّبة حسب الأثر على قيمة المنتج مقابل جهد التنفيذ

أولوية 1 إصلاح مخاطر سلامة البيانات — قبل كل شيء

- **حد أدنى لخصم الجلسات:** `Math.max(0, sessions - 1)` في `reduceBranchPlayersByOneSession()` يمنع الجلسات السالبة.
- **فلترة اللاعبين في التعويض:** استثناء `is_paused = true` واللاعبين المنتهية اشتراكاتهم من سلسلة التعويض.
- **استعلام bulk UPDATE بدل الحلقة:**

```
UPDATE players SET sessions = sessions + 1 WHERE branch_id = $1 AND is_paused = false AND
subscription_mode = 'حصص'
```

.

- **تفعيل رؤية الغيابات في لوحة التحكم:** جلب آخر 30 يومًا من الحضور فقط وتمريرها لـ `computeInsights()` .
- **تحويل `branch_id` في finance_tx لـ UUID FK:** أو على الأقل إضافة عمود `branch_name_snapshot TEXT` للحفظ التاريخي.

أولوية 2 عرض P&L لكل فرع — أعلى قيمة تجارية

إضافة عرض "لكل فرع" في صفحة المالية: الإيرادات، تكاليف الملعب، رواتب المدربين، وصافي الربح لكل فرع. البيانات موجودة كاملًا في `finance_tx` مفلترة بـ `branch_id` . لا تحتاج هذه الميزة جلب بيانات إضافية — فقط تحويل في طريقة عرض ما هو موجود. **هذا السؤال يُطرح يوميًا من كل مالك أكاديمية متعددة الفروع.**

أولوية 3 مسار تصرف من صفحة الرؤى

الرؤى تكشف "7 لاعبين تنتهي اشتراكاتهم." إضافة زر "تجديد الآن" يفتح modal الدفع مع تعبئة مسبقة للاعب، وزر رابط WhatsApp مباشر (`wa.me/{phone}?text=...`) لرسالة تذكير. هذا يحوّل الرؤية من قراءة إلى سير عمل. دالة `actions[]` موجودة في النوع — تحتاج فقط إلى واجهة.

أولوية 4 سجل المدفوعات لكل لاعب

عرض توسيعي أو لوحة جانبية في صفحة اللاعبين تُظهر التاريخ الكامل للمدفوعات (التاريخ، المبلغ، فترة الاشتراك). دالة `listPlayerPayments(playerId)` موجودة في `payments.ts` — تحتاج فقط إلى واجهة. يُلغي الحاجة للانتقال لصفحة المالية لمتابعة انتظام دفع اللاعب.

أولوية 5 تقرير رواتب شهري للطاقم

عرض مخصص يُظهر لكل موظف: الراتب الإجمالي، خصومات الغياب (كم جلسة)، وصافي الراتب. البيانات موجودة في `finance_tx` و `staff_attendance` . حاليًا هذه المعلومات موزعة على صفحتين مختلفتين بدون عرض مُجمّع.

أولوية 6 Server Components للصفحات الثقيلة

تحويل لوحة التحكم وصفحة المالية إلى React Server Components (أو استخدام `use server` مع Suspense) يُزيل: البياض الأولي عند التحميل، ازدواجية الطلبات، وتحميل بيانات كاملة على الجانب العميل. يُقلص وقت الاستجابة للمستخدم النهائي بشكل ملحوظ.

أولوية 7 تخزين الإشعارات في قاعدة البيانات

إضافة جدول `notifications` لحفظ الرؤيا الحرجة/التحذيرية عند توليدها. يُتيح: عدد الإشعارات غير المقروءة في الشريط الجانبي، مراجعة سجل التنبيهات التاريخية، وتدفق استلام/رفض. منطق الرؤية موجود بالكامل في `insights.ts` كدالة نقية — جاهز لهذا التحول.

07 التوصيات الاستراتيجية

كيف تتحول AcademyBase من نظام يعمل إلى منتج SaaS متميّز وذكي

S1 اعتماد مبدأ "مصدر الحقيقة الواحد" لحالة الاشتراك

النظام حاليًا يمتلك **ثلاثة تمثيلات مستقلة** لحالة اشتراك اللاعب: `players.end_date / sessions`، جدول `payments` عبر `subscription_end`، و `buildPeriodsMap()` في صفحة الحضور. هذه التمثيلات تتباعد صامتة. الإصلاح الاستراتيجي: جعل `payments` السجل المرجعي لكل فترات الاشتراك، وجعل `players.end_date` قيمة مُشتقة/مخزنة مؤقتًا تُحدَّث من المدفوعات فقط. هذا يضمن رؤية متطابقة لحالة الاشتراك عبر لوحة التحكم، المالية، والحضور.

S2 تحويل سلسلة الآثار الجانبية إلى event queue

سلسلة إلغاء التدريب (6 عمليات، لا transaction) هي أكثر منطقة خطرة في النظام. الحل المعماري: عند إلغاء تدريب، كتابة `session_event` في قاعدة البيانات، ومعالجته عبر Supabase Edge Function ذريًا. هذا يفصل الواجهة عن مزامنة المالية، يجعل الفشل الجزئي قابلًا للتعافي، ويوفر سجل تدقيق كامل بما تغيّر ولماذا.

S3 بناء طبقة تواصل مع اللاعبين

كل لاعب لديه رقم هاتف مُخزَّن. كل حدث في الأكاديمية (إلغاء تدريب، انتهاء اشتراك، تأخير دفع) يكتشفه النظام بالفعل. إضافة تكامل مع WhatsApp Business API أو SMS يحوّل AcademyBase من أداة إدارة إلى منصة تواصل — وهو المميّز الحقيقي للأكاديميات الصغيرة والمتوسطة التي تعمل بالقرب من عملائها.

S4 تجميع البيانات على جانب الخادم (Materialized Views)

صفحات الإحصائيات ولوحة التحكم تجلب بيانات خامة وتحسب كل شيء في JavaScript. مع النمو، هذا يكسر. الاستثمار الاستراتيجي هو Materialized Views في Postgres: `mv_monthly_revenue(academy_id, month, branch_id, amount)`، `mv_renewal_rate(academy_id, month, new_count, renew_count)`، `mv_attendance_rate(academy_id, month, branch_id, rate)` — تُحدَّث ليليًا. الصفحات تستعلم عن بيانات مُجمَّعة مسبقًا، مما يقلل الحمل بعامل 100x.

S5 إضافة سعة الفرع وإدارة قوائم الانتظار

الفروع حاليًا ليس لها حقل `max_players`. أكاديمية تُدير فرعًا بنسبة 95% لا تعرف ذلك من النظام. إضافة تتبع السعة يُتيح: إدارة قوائم انتظار اللاعبين الراغبين في الانضمام، تحذيرات السعة في محرك الرؤى، وإشارات تحسين التسعير (الفروع المكتملة تستحق أسعارًا أعلى).

S6 مفهوم "الموسم" — إدارة دورات الأكاديمية

النظام دائم لا يعرف الموسمية. كثير من الأكاديميات الرياضية تعمل بمواسم (سبتمبر – يونيو). إضافة جدول `seasons` يُتيح: ملخصات مالية لكل موسم، مقارنة موسم بموسم في لوحة التحكم، وعمليات "بداية موسم" مجمّعة (إعادة تعيين الجلسات، التسجيل الجماعي للاعبين العائدين).

S7 إكمال دور مدير الفرع

دور `branch_manager` مُطبَّق جزئيًا في قاعدة البيانات: له دور، صلاحية مالية اختيارية (`has_finance_access`)، وRLS محدود بالفرع. لكن من منظور المنتج، سير عمل مدير الفرع الفعلي (متابعة لاعبيه، حضورهم، ماليتهم) يتطلب تنقلًا عبر صفحات متعددة بدون تخصيص للفرع. لوحة تحكم مخصصة لمدير الفرع تحدّ جميع KPIs بفرعه ستجعل هذا الدور ذا قيمة حقيقية وتُمكّن المالكين من التفويض بأمان.

جدول الأولويات الموحّد

أهم 10 بنود مُرتّبة حسب الأثر على المنتج مقابل جهد التنفيذ

#	المشكلة	الأثر	الجهد	الأولوية
1	جلسات اللاعبين قد تصير سالبة (R7 + W7)	تلف بيانات	منخفض	فوري
2	اللاعبون الموقوفون/المنتهون يحصلون على تعويض (R1 + W2)	بيانات خاطئة	منخفض	فوري
3	رؤية الغيابات غائبة في لوحة التحكم (W4)	ميزة صامتة	منخفض	فوري
4	عرض P&L لكل فرع (UX1)	ضرورة تجارية	متوسط	هذا الأسبوع
5	finance_tx.branch_id نوع TEXT بلا FK (W3)	تلف تاريخي عند الحذف	متوسط	هذا الأسبوع
6	سلسلة إلغاء التدريب بلا atomicity (W1)	فشل جزئي صامت	عالٍ	هذا الشهر
7	سجل مدفوعات اللاعب في صفحة اللاعبين (UX5)	احتكاك يومي	متوسط	هذا الشهر
8	مسار تصرف من صفحة الرؤى (UX3)	رؤى بلا فعل	متوسط	هذا الشهر
9	لوحة التحكم تجلب كل المدفوعات التاريخية (P1)	أداء عند التوسع	متوسط	الربع القادم
10	تجميع البيانات على الخادم لصفحات الإحصائيات (P2 + S4)	قابلية التوسع	عالٍ	الربع القادم

الخلاصة: النظام مبني على أساس تقني متين وبنية RLS مدروسة لمنع التكرار. القضايا الثلاث الأولى في الجدول يمكن إصلاحها في ساعات قليلة وتمنع تلف بيانات فعلي. البنود 4–8 تمثل أعلى عائد على تجربة المستخدم. البنود 9–10 تضمن استمرارية الأداء مع النمو. الاستراتيجيات S1–S7 تضع الخارطة للقفزة من "نظام يعمل" إلى "منتج SaaS متميّز."